SYSTEM DESIGN

Chapter 52: Design a News Feed / Twitter

Personal Notes

If URL shortener is the "hello world" of system design, News Feed is the "real interview." It's the #2 most-asked system design problem (Twitter, Instagram, Facebook, TikTok, LinkedIn — every social/content platform hits this pattern). It's genuinely hard: a naive design breaks at scale, and the RIGHT design requires making a subtle trade-off (fan-out on write vs fan-out on read) that most first-time candidates miss.

This chapter walks through Twitter's design end-to-end. Along the way we'll tackle the two feed generation strategies, the CELEBRITY PROBLEM (what happens when Taylor Swift tweets to 200M followers?), the HYBRID approach that real systems use, feed ranking vs chronological ordering, and the scaling questions interviewers love. If you can design a news feed cleanly, you can design 80% of social/content products.

1. The Problem

Design a Twitter-like service. Users can post tweets, follow other users, and see a HOME TIMELINE — a chronological (or ranked) feed of tweets from everyone they follow. The core challenge is the timeline: efficiently generating it for millions of users, each following hundreds of accounts, each account posting many times per day.

```
    User Alice's follows:  [Bob, Carol, Dave, Eve, ...]

    User's home timeline = merged tweets from all follows,
                           sorted by time (or by ranking model)

    Bob:   "Hello world"      (2 min ago)
    Carol: "New coffee shop"  (5 min ago)
    Dave:  "Meeting notes"    (10 min ago)
    Eve:   "Vacation photo"   (12 min ago)
    ...
```

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Post a tweet (text, ~280 chars, optionally with media)
- Follow / unfollow a user
- Read the home timeline (tweets from users you follow)
- Read a user timeline (tweets from a specific user)
- (Optional) Search, retweets, likes, replies, DMs, hashtags

2.2 Non-Functional Requirements

- Scale — assume 300M total users, 200M DAU, average 200 follows/user
- Latency — home timeline load: p99 < 200 ms (this is the hot path)
- Availability — 99.9%+ uptime
- Consistency — EVENTUAL is fine for the feed (tweets can take a few seconds to propagate)
- Read:write ratio — very read-heavy (~100:1 or more)

2.3 Constraints to Confirm

- Which is the priority — post latency (fast tweet) or timeline load latency (fast feed)?
- Is timeline strictly chronological, or ranked by algorithm?
- How many tweets should a home timeline hold in cache — 500? 2000?
- How many followers does the biggest account have? (This determines the celebrity problem's severity.)

> **Interview tip:** The home timeline read latency is the single most important number. Users open Twitter and expect the feed instantly. If you optimize for anything else and leave the read path slow, you've built the wrong system.

3. Step 2 — Capacity Estimation

Tweets

```
200M DAU × 2 tweets/day/user = 400M tweets/day
÷ 86,400 seconds/day
= ~4,600 tweets/sec (average)
× 3 (peak factor)
= ~14,000 tweets/sec peak    ← write QPS
```

Timeline Reads

```
200M DAU × ~5 timeline loads/day = 1B loads/day
÷ 86,400 seconds/day
= ~11,600 reads/sec average
× 3 peak
= ~35,000 timeline reads/sec peak

Each timeline load returns ~50 tweets → each is expensive
without pre-computation.
```

Fan-Out Load (if push model)

```
4,600 tweets/sec × 200 followers/user = ~920,000 writes/sec

This is the write amplification if we PUSH every tweet to
every follower's timeline. Manageable, but with celebrities
```

```
    (200M followers), a single tweet = 200M fan-out writes. Ouch.
```

Storage

```
    400M tweets/day × 500 bytes (text + metadata) ≈ 200 GB/day
    Media (images/video) stored separately in blob store / CDN.

    Annual: ~75 TB/year of raw tweet data. Manageable.
```

4. Step 3 — API Design

```
POST /api/tweet
{
  "user_id": 12345,
  "content": "Hello, world!",
  "media_urls": ["s3://..."]           // optional
}
→ 201 { "tweet_id": 98765, "created_at": "..." }

_____________________________________________________________

GET /api/timeline/home?user_id=12345&limit=50&cursor=abc
→ 200 {
  "tweets": [...],
  "next_cursor": "def"
}

_____________________________________________________________

GET /api/timeline/user/:user_id?limit=50
→ 200 { "tweets": [...] }

_____________________________________________________________

POST /api/follow    { "follower_id": ..., "followee_id": ... }
POST /api/unfollow  { "follower_id": ..., "followee_id": ... }
```

Cursor-based pagination: Do NOT use offset/limit pagination for feeds. Offsets break when new tweets are inserted (users see duplicates or skip tweets). Cursor-based pagination (opaque cursor pointing to "last tweet_id seen") is stable across inserts. Every major feed uses this.

5. Step 4 — Data Model

```
Table: users
    user_id (PK)  |  username  |  display_name  |  followers_count  |  ...

Table: tweets
```

```
    tweet_id (PK)  |  user_id (FK)  |  content  |  created_at  |
media_urls

Table: follows
    follower_id  |  followee_id  |  created_at
    PRIMARY KEY (follower_id, followee_id)
    INDEX on (followee_id) — for "who follows me" queries

Table: user_timeline (cache/store)
    user_id  |  tweet_ids[] (list, sorted by time)
    — cached in Redis; also persisted for cold-start

Table: home_timeline (cache — the KEY data structure)
    user_id  |  tweet_ids[] (list, sorted by time, ~1000 entries)
    — LIVES IN REDIS; primary source of hot-timeline reads
```

The follows table can get huge: With 200M users following 200 on average, that's 40 BILLION rows. Shard by follower_id (for "my follows") and separately by followee_id (for "my followers") — two denormalized copies of the same data. Cheap in NoSQL, painful in SQL.

6. Step 5 — Feed Generation (The Core Challenge)

This is the beating heart of the design. TWO fundamental approaches, each with sharp trade-offs. Understanding this trade-off is what interviewers are really testing.

6.1 Approach A — Fan-Out on Write (Push Model)

When a user tweets, PUSH that tweet_id into the home_timeline cache of every follower.

```
    Alice tweets:
       1. Save tweet to tweets DB
       2. Look up Alice's followers (say, 500)
       3. For EACH follower's home_timeline in Redis:
             LPUSH home_timeline:{follower_id} tweet_id
       4. Done. Read is instant.

    When Bob reads his timeline:
       LRANGE home_timeline:{bob_id} 0 49  ← O(1), single Redis call
```

- Pros: reads are BLAZING FAST (single Redis lookup, ~1 ms)
- Pros: computation happens at write time (rare), off the hot path
- Cons: writes amplify by follower count — a tweet with 200M followers = 200M cache writes
- Cons: wasteful for inactive users (write to their timeline, they may never read it)

6.2 Approach B — Fan-Out on Read (Pull Model)

Only save the tweet. When Bob reads his timeline, QUERY each of the ~200 people he follows and merge their tweets on the fly.

```
Alice tweets:
   1. Save tweet to tweets DB
   2. Done. Cheap.

When Bob reads his timeline:
   1. Look up Bob's follows (~200 users)
   2. For each: fetch their recent tweets (SELECT LIMIT 50)
   3. Merge all 200 lists in-memory, sort by time
   4. Return top 50

Cost per read: expensive — 200+ DB queries, in-memory merge
```

- Pros: writes are cheap — one insert regardless of follower count
- Pros: works uniformly for celebrities and normal users
- Cons: reads are SLOW — hundreds of queries + merge per timeline load
- Cons: read latency is highly variable — depends on how many people you follow

6.3 Comparison

Aspect	Fan-Out on Write (Push)	Fan-Out on Read (Pull)
Read latency	~1 ms (single Redis)	100-500 ms (fan-out + merge)
Write cost	O(N) per tweet (N = followers)	O(1) per tweet
Best for	Most users (100-1000 followers)	Celebrities (10M+ followers)
Storage	High — timeline per user	Low — no timeline stored
Handles inactive users	Wastes work	Perfect — pay only on read
Handles new follows	Backfill needed	Automatic — reads live data

7. The Hybrid Approach — What Real Systems Use

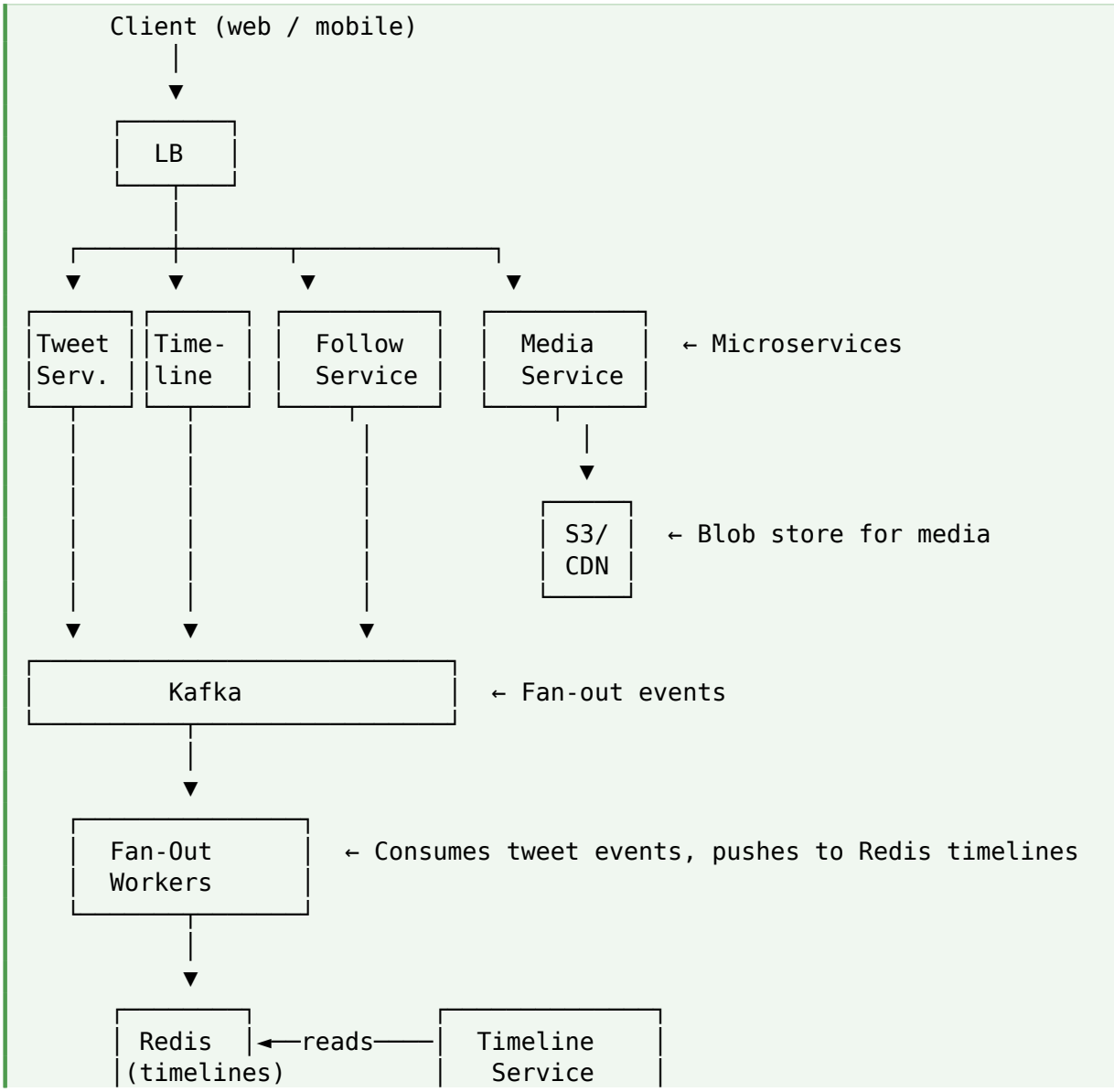
Neither pure approach works at Twitter's scale. The real answer is a HYBRID: use fan-out on write for normal users (fast reads for the 99%), and fan-out on read for celebrities (avoid the 200M write amplification).

```
Rule: if a user has < 1M followers  →  fan-out on WRITE
      if a user has >= 1M followers →  fan-out on READ

Bob reads his home timeline:
   1. Fetch pre-computed timeline from cache (populated by normal
follows)
   2. Fetch recent tweets from celebrity follows (Taylor Swift, Elon,
etc.)
```

```
        3. MERGE the two sources, sort by time
        4. Return top 50

    Only 1-5 celebrity queries per read → cheap.
    Normal 195 follows → no extra queries needed.
    Best of both worlds.
```

This is THE interview answer: "For most users, we fan out on write — pushing new tweets into each follower's Redis timeline. But for users above a threshold (say, 1M followers), we don't fan out at all. When any user reads their feed, we merge their pre-computed timeline with a live pull from the celebrities they follow." This shows both approaches AND the resolution — signals senior thinking.

8. Step 6 — High-Level Architecture



```
      └──────────┘              └──────────────────┘

     Persistent stores: Cassandra (tweets), MySQL sharded (users, follows)
```

Service Breakdown

- Tweet Service — creates tweets, publishes fan-out events to Kafka
- Timeline Service — assembles home timelines by merging cached + celebrity fetches
- Follow Service — manages the follows graph
- Media Service — handles uploads, stores in S3, distributes via CDN
- Fan-out Workers — consume Kafka events, push tweet_ids into follower timelines

9. Step 7 — Deep Dive: The Celebrity Problem

When Taylor Swift (with 100M+ followers) tweets, pure fan-out on write means 100M cache writes for ONE event. That's:

```
     100M writes to Redis
     Even at 100K writes/sec throughput per Redis node
     → 1000 seconds = 16+ minutes to fully propagate

     Meanwhile: real-time notifications, replies backing up, followers
     complaining they didn't see the tweet. Load spikes across the
cluster.

     Scale this to hundreds of celebrities tweeting/hour → system melts.
```

Mitigations

- HYBRID as above — don't fan-out at all for high-follower accounts
- If you MUST fan-out celebrities, prioritize ACTIVE followers only (people who logged in last 7 days)
- Rate-limit the fan-out — spread it over minutes rather than seconds
- Use a separate CELEBRITY tier of Redis, more replicas, more capacity

> **New follows are also problematic:** When Bob follows Alice, do we backfill Alice's recent tweets into Bob's timeline? If yes: extra work per follow. If no: Bob won't see Alice's older tweets until she posts new ones. Most services do a limited backfill (~50-100 recent tweets) at follow time.

10. Deep Dive: Timeline Ordering — Chronological vs Ranked

Chronological (Newest First)

- Simple: just sort by created_at descending
- Predictable: users know they'll see the latest
- Missed tweets: users may not see anything if they follow many active people

Ranked (Algorithmic Feed)

- Score each tweet by (recency, engagement, personalization, ad_boost)
- Uses ML models — predict likelihood of like/reply/retweet
- Better engagement (users stay longer) but less predictable
- Requires a scoring service in the read path — adds latency

```
// Simplified ranking model (in reality: gradient-boosted trees / neural
nets)
score = W1 * recency_score
      + W2 * predicted_engagement (like/reply/retweet)
      + W3 * relationship_strength (mutual follows, DMs, etc.)
      + W4 * content_quality_signal
      + W5 * ad_boost (for paid content)

// Twitter's chronological view mostly returns raw feed.
// The default "For You" tab runs each candidate tweet through the model.
```

For interviews: Even if you know ML ranking, keep it in a separate SERVICE that the timeline service calls. Don't tightly couple the read path to model inference. "Rank at the last mile" is the standard pattern — fetch candidates, then score, then return top N.

11. Deep Dive: Caching

Two Timeline Caches

- home_timeline:{user_id} — pre-computed, ~1000 tweet_ids for the user, LRU
- user_timeline:{user_id} — a user's own posted tweets, ~500 most recent

What Not to Cache

The FULL tweet content (text, media URLs, etc.) should be cached SEPARATELY from the timeline. Timelines store only tweet_IDs; on read, do a MULTIGET to fetch tweet content from a tweet cache (e.g., another Redis instance). This keeps timelines compact and lets tweet content be updated (deleted, edited) without touching every follower's timeline.

```
     Read flow:

     1. GET home_timeline:{bob_id}  →  [tweet_100, tweet_87,
tweet_45, ...]
     2. MGET tweet:100, tweet:87, tweet:45, ...  →  content, media,
user_info
     3. Optional: rank via model service
     4. Return top 50 to client

     Total: 2 Redis round-trips + optional ML call = ~10-30 ms
```

Cache Warming for Inactive Users

Users who haven't logged in for 30+ days probably won't in the next hour. Skip fan-out for them — save the compute. Rebuild their timeline on-demand when they return.

12. Deep Dive: Database Choices

Data	Store	Why
Tweets	Cassandra / DynamoDB	Write-heavy, key-value pattern by tweet_id
Users	MySQL (sharded)	Small table, needs relational queries
Follows	Cassandra / MySQL sharded	Denormalized; 2 tables (by follower, by followee)
Timelines	Redis (primary), Cassandra backup	Hot in-memory; persist for cold start
Media	S3 / Object storage + CDN	Large binary blobs, served directly to client
Analytics	Kafka → ClickHouse / BigQuery	Async pipeline, doesn't affect hot path

13. Deep Dive: Real-Time Updates

When Alice tweets and Bob is viewing his home page, should Bob see the tweet appear? Two ways:

- Poll — client re-fetches timeline every 30 seconds. Simple, but adds LOAD and latency.
- Push via WebSocket / SSE — server pushes new tweet_ids as they arrive.

```
Push flow:

1. Bob connects via WebSocket to a Notification Service
2. When Alice tweets, fan-out worker also publishes to a
   "realtime" topic in Kafka
3. Notification Service subscribes; when a tweet arrives for Bob's
   connected socket, it sends {tweet_id, preview} to Bob's client
4. Client shows "1 new tweet — click to see"

Bob's client fetches the tweet content on demand.
```

Push scales, but be careful: WebSocket connections are STATEFUL. 100M concurrent users = 100M open sockets across your servers. Use consistent hashing to route users to specific notification service instances. Twitter, Slack, and WhatsApp all use this pattern; expect gigantic connection pools.

14. Additional Features

14.1 Retweets

A retweet is essentially a POINTER to another tweet with an optional comment. Store retweet as a new tweet with a reference to the original — original's engagement counters increment. On the timeline, retweets appear as "Bob retweeted Alice's tweet".

14.2 Likes / Reactions

Two options: store LIKE events (heavy, but allows analytics like "who liked this") or store just counters (fast, but no per-user tracking). Twitter does both — likes table for the graph, denormalized counter on the tweet for display. Update counter async to avoid contention on hot tweets.

14.3 Replies / Threads

Each reply is a tweet with a parent_tweet_id. Build the thread by walking pointers. Cache resolved threads for viral posts.

14.4 Hashtags & Search

Async pipeline: extract hashtags from tweets, index in a search engine like Elasticsearch. Search queries hit ES separately from the timeline path.

14.5 Media Handling

Uploads go directly to S3 (client uploads via presigned URL, bypassing your servers). Serve via CDN. Do async processing (thumbnails, video transcoding) via a queue. Never put media in the main tweet path — it'll bottleneck everything.

15. Failure Modes

Failure	Mitigation
Redis timeline shard down	Serve stale timeline from Cassandra backup; degraded mode.
Fan-out worker lag	Users see delayed tweets. Auto-scale workers based on Kafka queue depth.
Tweet DB shard down	That shard's tweets unavailable temporarily. Read from replica; failover.
Celebrity spike (viral event)	Millions of extra reads/sec. Pre-scale during known events (elections, sports).
Kafka down	Fan-out queue cannot process; degrade to read-only mode; buffer writes locally.

16. Scaling Considerations

Sharding

- Tweets — shard by tweet_id (uniform distribution)
- Follows — DUAL sharding: by follower_id AND by followee_id (two copies)
- Timelines — shard by user_id in Redis (each timeline lives entirely on one node)

Read Replicas

Every hot store has replicas. Tweet DB shards have 2-3 replicas per region. Redis has replication for HA. Timeline reads can go to any replica; writes go to primary.

Multi-Region Deployment

For global users, run stacks in multiple regions (US, EU, APAC). Route users to their nearest region. Async cross-region replication for tweets — users in EU may see a US celebrity's tweet with ~1 second delay, which is acceptable.

CDN for Static Content

Media (images, video) served entirely from CDN — origin only responds on cache miss. This is the single biggest lever for reducing origin load in feeds.

17. Trade-offs Summary

Decision	Trade-off
Fan-out on write vs read	Fast reads vs cheap writes — HYBRID resolves both
Chronological vs ranked feed	Simplicity + predictability vs engagement + retention
Push vs poll for real-time	Push = fresh but stateful; poll = simple but stale
Denormalize follows or not	Denormalize for query speed; pay 2× storage
Backfill on follow vs not	Backfill = better UX; skip = cheaper on the follow path
Sync vs async likes	Sync = accurate counts; async = fast tweet + eventual consistency
Store media inline vs blob	Blob + CDN wins — always. Inline explodes DB size.

18. Common Mistakes

Mistake 1: Picking one fan-out approach without discussion

Saying "I'd fan out on write" without addressing the celebrity problem is a red flag. Interviewers WILL ask what happens with 100M followers. Bring up the hybrid approach yourself.

Mistake 2: Storing full tweet content in timeline cache

If tweets can be deleted or edited, every follower's timeline holds a stale copy. Store only tweet_IDs in timelines; fetch content from a separate tweet cache/store.

Mistake 3: Missing cursor-based pagination

Offset pagination breaks feeds (duplicates and skips as tweets arrive). Always use cursors — this is a mandatory design choice for any feed.

Mistake 4: Not addressing inactive users

Fanning out to every follower — even those inactive for months — wastes massive resources. Track active status; skip fan-out for cold users.

Mistake 5: Serving media through the app tier

Media must go S3 → CDN → client. Never route large binary content through your app servers or tweet DB. Presigned upload URLs bypass your infra on the write path too.

Mistake 6: Sync-updating like counters on hot tweets

A viral tweet might get 100K likes/second. Updating a single row 100K times/sec destroys your DB. Use async counter increments (Redis INCR + periodic flush, or a dedicated counter service).

Mistake 7: Ignoring ranking-vs-latency trade-off

ML-based ranking adds 50-200 ms to the read path. For interview purposes, mention that ranking happens at the last mile — fetch a candidate set (say 200 tweets), then rank the top 50. Don't score 10,000 tweets per read.

19. Best Practices

- Start with the read:write ratio and home timeline latency — those drive everything
- Explicitly discuss BOTH fan-out approaches before landing on the hybrid
- Use cursor-based pagination always for feeds
- Store only tweet_IDs in timelines; content in a separate tweet cache
- Skip fan-out for inactive users to save massive resources
- Async-update engagement counters (likes, retweets) — never sync
- Media via S3 + CDN — never inline
- Rank at the last mile — fetch candidates first, score after
- Multi-region deployment with async cross-region replication for global scale

20. Quick Reference

Concept	Meaning
Home timeline	Feed of tweets from users you follow
User timeline	Tweets posted by a specific user
Fan-out on write (push)	Push tweet to each follower's timeline on tweet
Fan-out on read (pull)	Assemble timeline by querying follows on read
Hybrid approach	Push for normal users, pull for celebrities
Celebrity problem	Fan-out explosion when a high-follower user tweets
Cursor pagination	Opaque cursor pointing to last-seen tweet — stable across inserts
Chronological feed	Sorted by created_at DESC — simple, predictable
Ranked feed	ML-scored tweets — better engagement, added complexity
Denormalized follows	2 copies of follows table — by follower AND by followee
Timeline cache	Redis; stores tweet_IDs only, not content
Tweet cache	Redis; stores tweet content, fetched via MGET
Fan-out worker	Kafka consumer that pushes tweets to follower timelines

21. Related Interview Problems

The News Feed pattern powers many other systems. Once you have this design internalized:

1. Design Instagram / Photo Feed — same fan-out pattern, media-heavy
2. Design TikTok / Short-Video Feed — ranked-only, ML-driven, cold-start problem
3. Design Facebook Feed — friend network, richer engagement signals
4. Design LinkedIn Feed — professional context, connection-based ranking
5. Design Reddit / Hacker News — community-scoped feeds, voting-based ranking
6. Design a Notification System — push architecture, delivery guarantees
7. Design a Live Comments / Chat overlay — sub-second delivery to large audiences
8. Design a Recommendation Feed (Netflix / YouTube) — pure ML-ranked, no explicit follows

22. Final Takeaway

News Feed is a pattern, not just a product. The core tension — cheap writes vs fast reads — appears in every social platform, every messaging app, every notification system. Master this design and you have the mental model for a huge swath of Google/Meta/LinkedIn system design interviews. The HYBRID approach is what separates strong candidates from average ones: showing you understand both extremes AND how to resolve the tension.

For interviews, memorize the flow: requirements (establish 100:1 read:write) → capacity (get fan-out numbers) → fan-out on write pros/cons → fan-out on read pros/cons → hybrid resolution → celebrity problem discussion → caching strategy (timeline IDs + tweet content separately) → real-time via push. If you can walk through this in 30-35 minutes, you've mastered the canonical social-feed design.

> **Key Takeaway:** News Feed = fan-out design problem. FAN-OUT ON WRITE: fast reads, expensive writes for celebrities. FAN-OUT ON READ: cheap writes, slow reads. HYBRID (real answer): push for normal users, pull for celebrities, merge on read. Store timeline IDs only, tweet content separately. Use CURSOR-based pagination. SKIP fan-out for inactive users. Async engagement counters. Media via S3+CDN. Rank at the last mile. Push via WebSocket for real-time. The celebrity problem is the interview crux — always address it.

23. What's Next?

Now that you've done the two canonical designs (URL shortener, news feed), branch out:

- Design a Chat System (WhatsApp/Slack) — WebSocket, presence, delivery guarantees, group chats
- Design a Rate Limiter — token bucket implementation at scale, distributed coordination
- Design a Distributed Cache — Redis internals, consistent hashing, replication, eviction
- Design Uber / Lyft — geohashing, driver-rider matching, real-time location tracking
- Design YouTube — video encoding pipeline, adaptive bitrate, CDN edge caching
- Design a Notification System — push vs pull, priority queues, template management
- Deep dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep dive: Kafka Architecture — partitions, consumer groups, exactly-once semantics